11.Travelling Salesman Problem (TSP) Using Exhaustive Search

AIM

To write a Python program to solve the **Travelling Salesman Problem (TSP)** using the Exhaustive Search technique by finding the shortest possible route that visits every city exactly once and returns to the starting city.

PROCEDURE

1. Import the `permutations` function from the `itertools` module.
2. Create a cost matrix representing the distances between different cities.
3. Define a function to solve the Travelling Salesman Problem.
4. Select the starting city and generate all possible permutations of the remaining cities.
5. Construct a complete route by adding the starting city at the beginning and end.
6. Calculate the total travel cost for each possible route.
7. Compare the calculated cost with the minimum cost.
8. Update the minimum cost and shortest path whenever a shorter route is found.
9. Repeat the process until all possible routes are evaluated.
10. Display the shortest path and its minimum travel cost.

main.py | Visualize | Run ▶ | Output

```python
from itertools import permutations
def tsp(graph, start):
    n = len(graph)
    cities = list(range(n))
    cities.remove(start)
    min_cost = float('inf')
    best_path = None
    for path in permutations(cities):
        current_path = [start] + list(path) + [start]
        cost = 0
        for i in range(len(current_path) - 1):
            cost += graph[current_path[i]][current_path[i + 1]]
        if cost < min_cost:
            min_cost = cost
            best_path = current_path
    return best_path, min_cost
graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
```

```
Shortest Path: [0, 1, 3, 2, 0]
Minimum Cost: 80
```

RESULT

Thus, the Python program successfully solves the **Travelling Salesman Problem using the Exhaustive Search technique** by evaluating all possible routes and identifying the route with the minimum travel cost.

Shortest Path: 0 → 1 → 3 → 2 → 0

Minimum Cost: 80